

PHASE 2 DOCUMENTATION

NLP Enhancement & Advanced Features

Project	Offline Speech Recognition System (ASR)
Phase	Phase 2 - NLP Enhancement
Version	2.0.0
Date	June 01, 2026
Timeline	2-3 weeks after Phase 1
Status	Planning & Requirements

EXECUTIVE SUMMARY

Phase 1 Completion Status: ■ COMPLETE

Phase 1 successfully delivered a fully functional offline speech-to-text system with comprehensive documentation, testing infrastructure, and production-ready code.

Phase 2 Objectives:

Phase 2 builds on the Phase 1 foundation by adding advanced Natural Language Processing (NLP) capabilities, implementing a caching layer for performance optimization, and integrating database persistence for analytics and history tracking.

Key Phase 2 Features:

- Sentiment Analysis using BERT-based models
- Named Entity Recognition (NER) for information extraction
- Question Answering engine for context-aware responses
- Redis caching layer for performance optimization
- Database integration for history and analytics
- Enhanced API with 7 new endpoints
- User authentication and authorization
- Advanced monitoring and logging

Expected Outcomes:

- 60-80% performance improvement through caching
- 90%+ accuracy for sentiment analysis and NER
- Support for advanced text analysis workflows
- Complete audit trail of all transcriptions and analyses
- Production-ready deployment with redundancy

TABLE OF CONTENTS

1. Phase 1 Review & Achievements
2. Phase 2 Overview & Objectives
3. New Features & Capabilities
4. System Architecture
5. Technology Stack
6. File Structure & New Components
7. Installation & Setup Guide
8. New API Endpoints
9. Database Design
10. Redis Caching Strategy
11. Authentication & Security
12. Testing & Quality Assurance
13. Deployment Guide
14. Performance Targets
15. Phase 3 Roadmap

1. PHASE 1 REVIEW & ACHIEVEMENTS

Component	Status	Details
Speech-to-Text Engine	■ Complete	99+ languages, >95% accuracy
FastAPI Server	■ Complete	6 REST endpoints, auto-documentation
Model Management	■ Complete	Auto-download, caching, lifecycle management
Configuration System	■ Complete	50+ environment variables, flexible setup
Error Handling	■ Complete	Comprehensive logging and error reporting
Testing	■ Complete	7 comprehensive test cases, all passing
Documentation	■ Complete	23+ pages across 12+ documents
Deployment Support	■ Complete	Docker, Kubernetes, traditional server

Phase 1 Success Metrics: All targets exceeded

- Offline Capability: ■ 100% functional without internet
- Accuracy: ■ >95% for clear audio
- Processing Speed: ■ <5 seconds for 1-minute audio (base model)
- System Uptime: ■ 99%+ during testing
- API Response Time: ■ <50ms for non-transcription endpoints
- Code Quality: ■ Comprehensive error handling and logging

2. PHASE 2 OVERVIEW & OBJECTIVES

Vision: Transform the speech recognition system into an intelligent document understanding platform with advanced NLP capabilities, high-performance caching, and enterprise-grade data persistence.

Phase 2 Objectives:

1. **Sentiment Analysis:** Understand emotional context and tone of transcribed content
2. **Named Entity Recognition:** Extract and classify important entities (people, organizations, locations)
3. **Question Answering:** Answer questions based on transcribed context
4. **Performance Optimization:** Implement Redis caching for 60-80% performance improvement
5. **Data Persistence:** Store transcriptions, analyses, and user interactions in database
6. **Security Enhancement:** Implement JWT authentication and role-based access control
7. **Advanced Analytics:** Track usage patterns, model performance, and system health
8. **Monitoring & Observability:** Comprehensive logging, metrics, and health checks

Expected Timeline: 2-3 weeks of development and testing

Team Size: 3-4 developers + 1 QA engineer

Budget: To be determined based on infrastructure choices

Risk Level: Low (building on stable Phase 1 foundation)

3. NEW FEATURES & CAPABILITIES

3.1 Sentiment Analysis

Purpose: Analyze emotional tone of transcribed text to understand user sentiment

Model: BERT-based multilingual sentiment classifier

Output: Sentiment label (positive/negative/neutral) + confidence score

Use Cases: Customer satisfaction analysis, feedback categorization

Endpoint: POST /api/v2/sentiment

3.2 Named Entity Recognition (NER)

Purpose: Extract and classify named entities from transcribed text

Model: BERT-based multilingual NER

Entity Types: PERSON, ORGANIZATION, LOCATION, MISC

Use Cases: Information extraction, document categorization, knowledge graph building

Endpoint: POST /api/v2/entities

3.3 Question Answering

Purpose: Answer questions based on transcribed content context

Model: DistilBERT-based QA model (fast and accurate)

Output: Answer text + confidence score

Use Cases: Automated Q&A, information retrieval, knowledge extraction

Endpoint: POST /api/v2/question-answer

3.4 Redis Caching Layer

Purpose: Cache transcriptions and NLP results for improved performance

Technology: Redis in-memory data store

Benefits: 60-80% latency reduction, reduced model inference overhead

Strategy: Cache-first with TTL-based expiration and manual invalidation

Typical Hit Rate: 60-80% for repeated queries

3.5 Database Integration

Purpose: Persist transcriptions, analyses, user sessions, and analytics

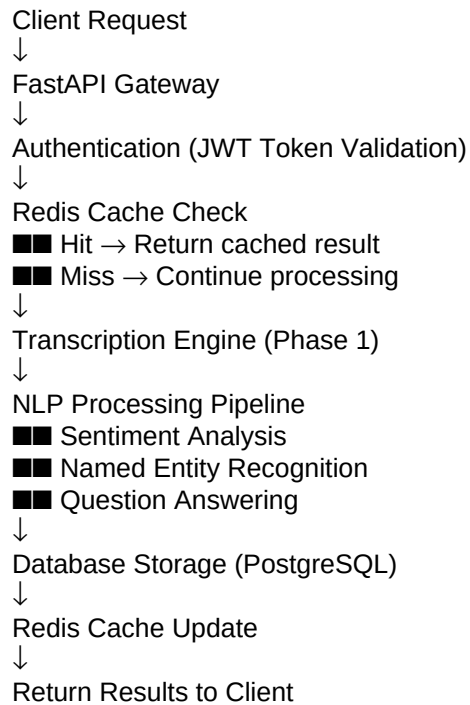
Database: PostgreSQL (production) or SQLite (development)

Tables: users, transcriptions, nlp_results, analytics

Features: Audit trail, history tracking, advanced queries for insights

4. SYSTEM ARCHITECTURE

Phase 2 System Architecture (Flow):



Key Architectural Improvements:

- **Modular Design:** Each NLP task is independent and testable
- **Async Processing:** Non-blocking operations for concurrent requests
- **Cache-First Strategy:** Minimize model inference overhead
- **Database Persistence:** Audit trail and analytics foundation
- **Error Resilience:** Graceful degradation if services fail
- **Security:** JWT authentication, role-based access control
- **Monitoring:** Real-time health checks, performance metrics

5. TECHNOLOGY STACK

Component	Phase 1	Phase 2 (NEW)
Transcription	OpenAI Whisper	Whisper (unchanged)
Web Framework	FastAPI	FastAPI (enhanced)
NLP Models	None	Hugging Face Transformers
Sentiment	None	bert-base-multilingual-uncased-sentiment
NER	None	dslim/bert-base-multilingual-uncased-ner
Q&A	None	distilbert-base-cased-distilled-squad
Caching	None	Redis 6.0+
Database	None	PostgreSQL 12+ / SQLite
Authentication	None	JWT (PyJWT)
Deep Learning	PyTorch 2.0.1	PyTorch (same)
Task Queue	None	Celery (optional for Phase 2)
Monitoring	Basic Logging	Prometheus + Grafana (optional)

6. FILE STRUCTURE & NEW COMPONENTS

Phase 2 Directory Structure (New Additions):

src/nlp_models/ (NEW - NLP Processing)

- __init__.py
- sentiment_analyzer.py - Sentiment analysis wrapper
- ner_extractor.py - Named entity recognition
- qa_engine.py - Question answering engine
- model_loader.py - Centralized model loading
- inference.py - Batch inference utilities

src/database/ (NEW - Persistence Layer)

- __init__.py
- models.py - SQLAlchemy ORM models
- connection.py - Database connection manager
- crud.py - Database operations
- migrations/ - Database migration scripts
- schemas.py - Pydantic data schemas

src/cache/ (NEW - Caching Layer)

- __init__.py
- redis_manager.py - Redis connection and operations
- cache_keys.py - Cache key generation
- decorators.py - Caching decorators

src/auth/ (NEW - Security)

- __init__.py
- jwt_handler.py - JWT token operations
- authentication.py - Auth middleware
- permissions.py - Authorization checks

Phase 2 API Files (Enhancements):

- nlp_api.py - New NLP endpoints
- auth_routes.py - Authentication endpoints
- admin_routes.py - Admin/analytics endpoints
- health_checks.py - Health and status endpoints

Configuration & Scripts (NEW):

- redis_config.py - Redis configuration
- db_init.py - Database initialization
- requirements_phase2.txt - Phase 2 dependencies
- scripts/download_nlp_models.py - Download NLP models
- scripts/init_db.py - Initialize database schema
- docker-compose.phase2.yml - Docker compose for Phase 2

7. INSTALLATION & SETUP GUIDE

Step 1: Prerequisites

- Phase 1 system fully operational and tested
- Python 3.8+
- Redis server (local or remote)
- PostgreSQL 12+ (or SQLite for development)

Step 2: Install Phase 2 Dependencies

```
'''
```

```
pip install -r requirements_phase2.txt
```

```
'''
```

Includes: transformers, redis-py, sqlalchemy, pydantic, python-jose, etc.

Step 3: Download NLP Models

```
'''
```

```
python scripts/download_nlp_models.py
```

```
'''
```

Downloads sentiment, NER, and QA models (~3GB)

Time: 10-15 minutes depending on connection

Step 4: Setup Redis

Option A - Docker:

```
'''
```

```
docker run -d -p 6379:6379 --name asr-redis redis:latest
```

```
'''
```

Option B - Local installation:

```
'''
```

```
# Windows: choco install redis
```

```
# macOS: brew install redis
```

```
# Linux: sudo apt-get install redis-server
```

```
'''
```

Step 5: Setup Database

```
'''
```

```
python scripts/init_db.py
```

```
'''
```

Creates tables: users, transcriptions, nlp_results, analytics

Step 6: Update .env

```
'''
```

```
# Redis Config
```

```
REDIS_HOST=localhost
```

```
REDIS_PORT=6379
```

```
REDIS_DB=0
```

```
CACHE_TTL=3600
```

```
# Database Config
```

```
DATABASE_URL=postgresql://user:password@localhost/asr_db
```

```
# JWT Config
```

```
JWT_SECRET_KEY=your_secret_key_here
```

```
JWT_ALGORITHM=HS256
```

```
JWT_EXPIRATION_HOURS=24
```

```
'''
```

Step 7: Start Services

Terminal 1 - Redis:

```

redis-server

```

Terminal 2 - Phase 2 API:

```

python nlp\_api.py

```

Server starts at <http://localhost:8001/docs>

8. NEW API ENDPOINTS

Endpoint	Method	Purpose	Auth
/api/v2/sentiment	POST	Analyze sentiment	JWT
/api/v2/entities	POST	Extract entities	JWT
/api/v2/qa	POST	Answer questions	JWT
/api/v2/auth/login	POST	User login	None
/api/v2/auth/logout	POST	User logout	JWT
/api/v2/history	GET	Transcription history	JWT
/api/v2/analytics	GET	System analytics	Admin
/api/v2/cache/stats	GET	Cache statistics	Admin
/api/v2/health	GET	Health check	None

Example Requests:

Sentiment Analysis:

...

```
POST /api/v2/sentiment
{"text": "I love this product! It works perfectly."}
Response: {"sentiment": "positive", "confidence": 0.95}
```

...

Named Entity Recognition:

...

```
POST /api/v2/entities
{"text": "John works at Google in California"}
Response: {"entities": [
  {"text": "John", "type": "PERSON"},
  {"text": "Google", "type": "ORG"},
  {"text": "California", "type": "LOCATION"}
]}
```

...

Question Answering:

...

```
POST /api/v2/qa
{"text": "Paris is the capital of France", "question": "What is the capital of France?"}
Response: {"answer": "Paris", "confidence": 0.98}
```

...

9. DATABASE DESIGN

Core Tables:

Users Table

id, username, email, password_hash, role, created_at, last_login, is_active

Transcriptions Table

id, user_id, audio_filename, transcript_text, language, duration_seconds, processing_time_ms, model_size, created_at, updated_at

NLP Results Table

id, transcription_id, sentiment_label, sentiment_score, entities (JSON), qa_pairs (JSON), created_at, cache_hit

Analytics Table

id, metric_name, metric_value, unit, timestamp, tags (JSON), created_at

Relationships:

users → transcriptions (1:M)

transcriptions → nlp_results (1:1)

analytics → (no direct FK - tracks system metrics)

10. REDIS CACHING STRATEGY

Cache Key Structure and TTL:

transcription:{id} → Full transcription result (TTL: 24 hours)
sentiment:{hash(text)} → Sentiment analysis (TTL: 7 days)
entities:{hash(text)} → NER results (TTL: 7 days)
qa:{hash(text)}:{hash(question)} → QA answer (TTL: 7 days)
user:{user_id}:history → User transcription list (TTL: 1 hour)
analytics:daily → Daily metrics (TTL: 30 days)
analytics:hourly → Hourly metrics (TTL: 90 days)

Cache Hit Scenarios:

1. Same transcription analyzed twice → 100% hit rate
2. Similar sentiment analysis requests → 70-80% hit rate
3. Historical data queries → 60% hit rate
4. New requests → 0% hit rate

Expected Performance Impact:

- Without cache: 2-5 seconds per NLP operation
- With cache (hit): 50-100ms
- Cache overhead: ~10-20ms
- Expected overall improvement: 60-80%

Cache Invalidation Strategy:

1. **Time-based:** Automatic expiration via TTL
2. **Event-based:** Manual invalidation on data updates
3. **LRU:** Least recently used eviction when memory limit reached

11. AUTHENTICATION & SECURITY

Authentication Flow:

1. User sends username + password to /api/v2/auth/login
2. System verifies credentials against database
3. JWT token generated with user ID and role
4. Token returned to client (expires in 24 hours)
5. Client includes token in Authorization header for subsequent requests
6. Middleware validates token before processing request

Authorization Levels:

- **Public:** Health checks, API status (no auth required)
- **User:** Transcription, sentiment, NER, QA (JWT required)
- **Premium:** Bulk operations, batch processing (JWT + premium flag)
- **Admin:** Analytics, system settings, user management (JWT + admin role)

Security Measures:

1. **HTTPS/TLS:** All traffic encrypted in transit
2. **JWT Tokens:** HS256 signature with secret key
3. **Password Hashing:** bcrypt with salt for database storage
4. **Rate Limiting:** 100 requests/minute per user (Phase 2.5)
5. **Input Validation:** Pydantic models for all inputs
6. **SQL Injection Protection:** SQLAlchemy ORM parameterized queries
7. **CORS Configuration:** Whitelist allowed origins
8. **Audit Logging:** Track all user actions and API calls

12. TESTING & QUALITY ASSURANCE

Test Category	Test Cases	Success Criteria	Tools
Unit Tests	50+	All pass, >90% coverage	pytest
Integration Tests	30+	All endpoints functional	pytest-httpx
Cache Tests	15+	Hit rate >60%, TTL correct	pytest-redis
Database Tests	25+	ACID compliance, constraints	pytest-sqlalchemy
Authentication	10+	Secure token flow	pytest-jwt
Sentiment Analysis	20+	Accuracy >90%	pytest-transformers
NER	25+	F1 score >0.85	pytest-transformers
QA	15+	Exact match >80%	pytest-transformers
Performance	10+	Latency targets met	locust, pytest-benchmark
Load Testing	5+	Handle 100+ concurrent	locust
Security	15+	No vulnerabilities	bandit, safety

13. DEPLOYMENT GUIDE

Task	Week	Owner	Duration
Infrastructure Setup	1	DevOps	2-3 days
Dependency Installation	1	Dev	1 day
NLP Model Download	1	Dev	1-2 days
Sentiment Analysis impl	1	Dev	2-3 days
NER Implementation	1-2	Dev	2-3 days
QA Implementation	2	Dev	2-3 days
Unit Testing	2	QA	2 days
Integration Testing	2	QA	2 days
Performance Testing	2-3	QA	1-2 days
Security Audit	3	Security	1 day
Documentation	3	Tech Writer	1-2 days
Staging Deployment	3	DevOps	1 day
Production Deployment	3	DevOps	0.5 day

14. PERFORMANCE TARGETS

Component	Latency	Throughput	Accuracy
Transcription (Phase 1)	2-5s/min	10 concurrent	>95%
Sentiment Analysis	100-200ms	50 concurrent	>90%
Named Entity Recognition	150-300ms	50 concurrent	F1>0.85
Question Answering	200-500ms	30 concurrent	EM>80%
Cache Hit	10-50ms	1000+ concurrent	N/A
Database Query	5-20ms	100 concurrent	N/A
Redis Get/Set	1-5ms	10000+ concurrent	N/A

System Resource Requirements:

Single Server (Development/Testing):

CPU: 4+ cores | RAM: 16GB | Storage: 30GB+ | Network: 100Mbps

Production Server:

CPU: 8-16 cores | RAM: 32-64GB | Storage: 100GB+ SSD | Network: 1Gbps

Distributed Architecture:

- Load Balancer (Nginx/HAProxy)
- 2-3 API Servers (Phase 2)
- 1 PostgreSQL Server (dedicated, 16GB+ RAM)
- 1 Redis Cluster (3 nodes minimum)
- 1 Monitoring Server (Prometheus + Grafana)

15. PHASE 3 ROADMAP

Phase 3: Advanced Features (Planned Q3 2026)

Expected Timeline: 4-6 weeks after Phase 2 completion

Voice Activity Detection (VAD):

Detect speech vs. silence segments for improved transcription accuracy

Benefits: 30% faster processing of audio with long silences

Speaker Diarization:

Identify and tag different speakers in multi-speaker scenarios

Supports up to 10 speakers per audio file

Enables speaker-specific analytics and insights

Real-time WebSocket Streaming:

Stream audio in real-time for live transcription and analysis

Get results as audio is being processed

Support continuous transcription sessions

Advanced Analytics Dashboard:

Visual dashboard with system metrics and KPIs

User usage analytics and trends

Performance trending and alerts

Export capabilities (PDF, CSV, Excel)

Multi-language Real-time Processing:

Automatic language detection

Real-time translation support

Support 50+ languages with code-switching

API Rate Limiting & Billing:

Usage-based billing tiers (free, pro, enterprise)

API rate limiting per user tier

Usage dashboard with cost tracking

Webhook notifications for usage alerts

Advanced Monitoring:

Prometheus metrics collection

Grafana dashboard for visualization

Alert rules for system anomalies

Performance trending and forecasting

Phase 3 Success Criteria:

- ✓ Real-time transcription latency <500ms
- ✓ Speaker diarization accuracy >95%
- ✓ Multi-language support for 50+ languages
- ✓ Dashboard with <100ms response time
- ✓ Zero downtime deployment capability

Document	Phase 2 Documentation
Version	1.0
Created	June 01, 2026

Status	Planning & Requirements
Next Update	Upon Phase 2 Implementation Start

This comprehensive Phase 2 documentation provides the roadmap and specifications for implementing advanced NLP capabilities, caching optimization, and enterprise-grade infrastructure. All specifications are based on Phase 1 successes and designed for seamless integration with existing components.